PYXCEL
AI-POWERED SPREADSHEETSYSTEM

Integrated with Ollama LLaMA for Offline Intelligence

TEAM ID : PY028

TEAM NAME : TECHXCEL

TEAM MEMBERS :

GEETHA. C

JAGATEESHWARI. P

DOMNIC BIYASHA A

HARUN B

JEGATHEESH S

JESINTH M

Introduction

In today's digital world, data has become an essential part of decision-making in fields such as education, business, finance, and research. Many organizations and individuals rely on spreadsheet applications to store and analyze data because they are easy to use and widely available. One of the most commonly used spreadsheet tools is Microsoft Excel, which provides features for calculations, data organization, and visualization. However, performing detailed analysis in spreadsheets often requires manual effort and a good understanding of formulas and functions. For many users, especially beginners, this process can be time-consuming and sometimes confusing.

Researchers have also found that manual spreadsheet operations can lead to errors when working with large datasets. According to studies by Raymond R. Panko, spreadsheet mistakes are quite common and may affect the accuracy of results, especially when data analysis is done manually. As the size of data continues to grow, relying only on manual spreadsheet operations becomes inefficient and increases the chances of errors.

With the advancement of modern programming tools, automated data analysis has become more accessible. The programming language Python has gained popularity in data science because it provides powerful libraries that simplify data processing tasks. Libraries such as Pandas help in organizing and analyzing large datasets efficiently, while OpenPyXL allows developers to read and modify Excel files programmatically. According to Wes McKinney, these tools significantly improve the speed and efficiency of data analysis compared to traditional spreadsheet methods.

In addition, the integration of Artificial Intelligence in data analysis systems has created new opportunities for generating useful insights from raw data. AI technologies can automatically identify patterns, detect unusual values, and provide explanations for data trends. Research by Stuart Russell and Peter Norvig highlights that intelligent systems can support better decision-making by simplifying complex data analysis tasks.

Based on these developments, this research introduces **PYXCEL Automator**, a spreadsheet automation tool designed to simplify Excel data processing. The system combines Python-based data analysis techniques with a user-friendly interface to help users analyze spreadsheet data, detect anomalies, and generate insights more efficiently. The goal of this system is to reduce manual work, minimize errors, and make data analysis easier for both technical and non-technical users.

2. Problem Statement

Spreadsheets are widely used for managing and analyzing data in many areas such as education, business, and research. Many users depend on spreadsheet applications like Microsoft Excel to organize information, perform calculations, and generate reports. Although these tools provide several useful features, many data processing tasks still need to be done manually. Users often spend a significant amount of time performing operations such as cleaning data, applying formulas, and analyzing results.

As described in the project documentation, traditional spreadsheet usage requires users to have knowledge of formulas and analytical functions in order to perform accurate calculations and extract meaningful insights from data . For users who are not familiar with these functions, performing data analysis can be challenging and time-consuming. In addition, when working with large datasets,

manual operations increase the chances of errors, which can lead to incorrect results and poor decision-making.

Another major problem is the difficulty of identifying anomalies or unusual patterns in large spreadsheets. Detecting such patterns manually requires careful examination of data, which is not always practical when the dataset is large. Furthermore, extracting useful insights from raw spreadsheet data often requires expertise in data analysis, which many users may not possess.

These challenges highlight the need for an automated system that can simplify spreadsheet operations and reduce the manual workload involved in data processing. By using programming technologies such as Python and data processing libraries like Pandas, it is possible to automate many spreadsheet tasks and improve the efficiency of data analysis.

Therefore, the main problem addressed in this project is the lack of an easy-to-use system that can automatically analyze spreadsheet data, detect anomalies, and generate meaningful insights without requiring advanced technical knowledge. To solve this problem, the PYXCEL Automator system is proposed as an intelligent spreadsheet automation tool that simplifies Excel data analysis and improves productivity by reducing manual effort .

3. Objectives

The PyXcel project is guided by the following primary and secondary objectives:

Primary Objectives

• Replace VBA/macro workflows with natural language instructions processed by a local LLM, eliminating the need for programming knowledge.

• Enable offline AI inference using Ollama and LLaMA, ensuring all data remains on the user's local machine at all times.

• Provide a modern, accessible graphical user interface built with PySide6 that is intuitive for non-technical users.

• Integrate directly with existing PyXcel backend modules (calculation, data_cleaner, vlookup, merger, visualizer) without disrupting the existing codebase.

• Deliver a production-ready system that can be launched with a single double-click via an automated batch launcher. 3.2 Secondary Objectives • Generate accurate Excel formulas from plain-English descriptions, including modern functions such as XLOOKUP, FILTER, and LET. • Automatically detect and compute Key Performance Indicators (KPIs) from any uploaded spreadsheet without requiring manual configuration. • Support multi-turn conversational analysis of spreadsheet data, allowing users to ask follow-up questions and receive contextually aware responses. • Provide an AI-powered data cleaning pipeline that applies pandas transformations based on user instructions. • Maintain a clean migration path from the existing Tkinter GUI to the new PySide6 GUI, preserving all backend logic during transition.

3. Objectives

The **PyXcel Automator project** is designed to simplify spreadsheet operations and make data analysis easier for users. The objectives of the project are divided into **primary objectives** and **secondary objectives**.

3.1 Primary Objectives

1. **Simplify spreadsheet operations**
   Replace complex VBA or macro-based workflows with simple natural language instructions so that users can perform tasks without programming knowledge.

2. **Ensure secure offline processing**
   Enable AI processing locally using tools such as Ollama and LLaMA so that all user data remains on the local machine.

3. **Provide an easy-to-use interface**
   Develop a modern and user-friendly graphical interface using PySide6 so that even non-technical users can easily operate the system.

4. **Integrate with existing modules**
   Connect the system with existing backend modules like calculation, data cleaning, lookup, merging, and visualization without changing the core structure.

5. **Easy system launch**
   Make the application simple to run through an automated launcher that allows the system to start with a single double-click.

3.2 Secondary Objectives

1. **Automatic formula generation**
   Convert plain English instructions into Excel formulas such as XLOOKUP, FILTER, and LET.

2. **Automatic KPI detection**
   Identify and calculate important Key Performance Indicators (KPIs) from uploaded spreadsheets.

3. **Conversational data analysis**
   Allow users to ask questions about spreadsheet data and receive meaningful responses.

4. **AI-based data cleaning**
   Provide automated data cleaning using libraries like Pandas.

5. **Smooth GUI transition**
   Ensure a smooth migration from the older Tkinter

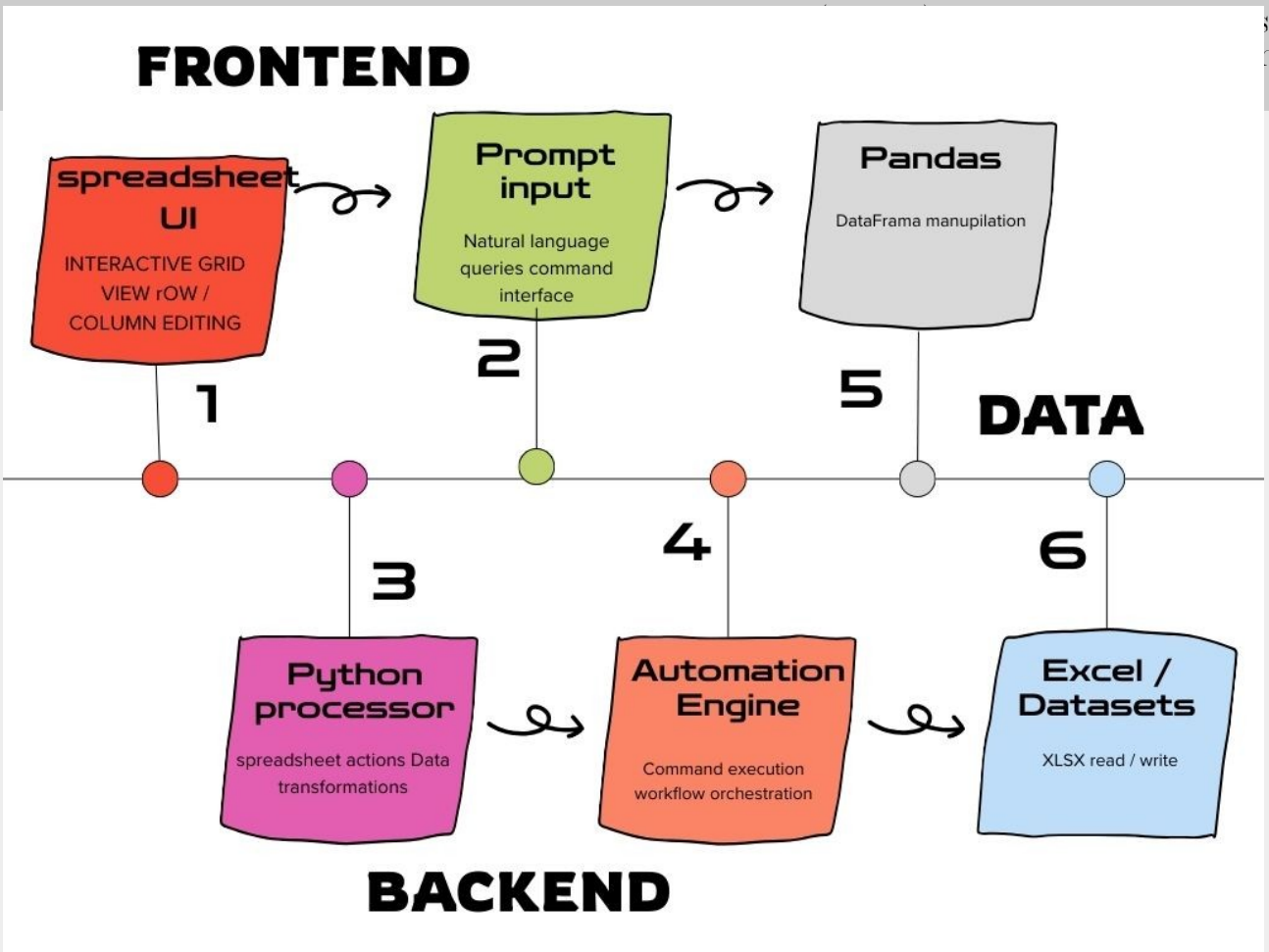
interface to the new PySide6 interface while keeping existing functionalities intact.

4. System Architecture

PyXcel follows a three-tier layered architecture with a local AI inference engine. The following describes each layer and their interactions.

Here is the information in **tabular format**:

Layer	Component	Responsibility
Presentation	PySide6 GUI	Provides the user interface, tab navigation, AI panels, KPI card rendering, and chat interface for user interaction.
Agent API	Flask REST Server	Handles requests from the GUI, manages communication with the AI model, and returns results through the local server (localhost:5000).
AI Inference	Ollama + LLaMA 3.1	Performs local AI model processing, generates context-aware code, and analyzes spreadsheet data.
Processing	openpyxl + pandas	Inspects Excel workbooks, executes data processing operations, performs data transformations, and manages file input/output.
Storage		sets, ne



Data Flow

When a user enters a natural language instruction, the system follows a series of steps to process it. First, the instruction from the **PySide6 GUI** is sent to the backend server using Flask along with the Excel file name. The system then reads the workbook and extracts important details such as sheet names, column headers, and sample data to understand the spreadsheet structure.

Next, this information is combined with the user's instruction and sent to the local AI model through Ollama. The LLaMA 3.1 model analyzes the request and generates an appropriate response, such as Python code using openpyxl, an Excel formula, KPI data, or a text explanation.

Finally, the backend processes the output, applies any required changes to the Excel file, and sends the result back to the GUI, where it is displayed as formulas, KPI cards, results, or chat responses for the user.

Methodology

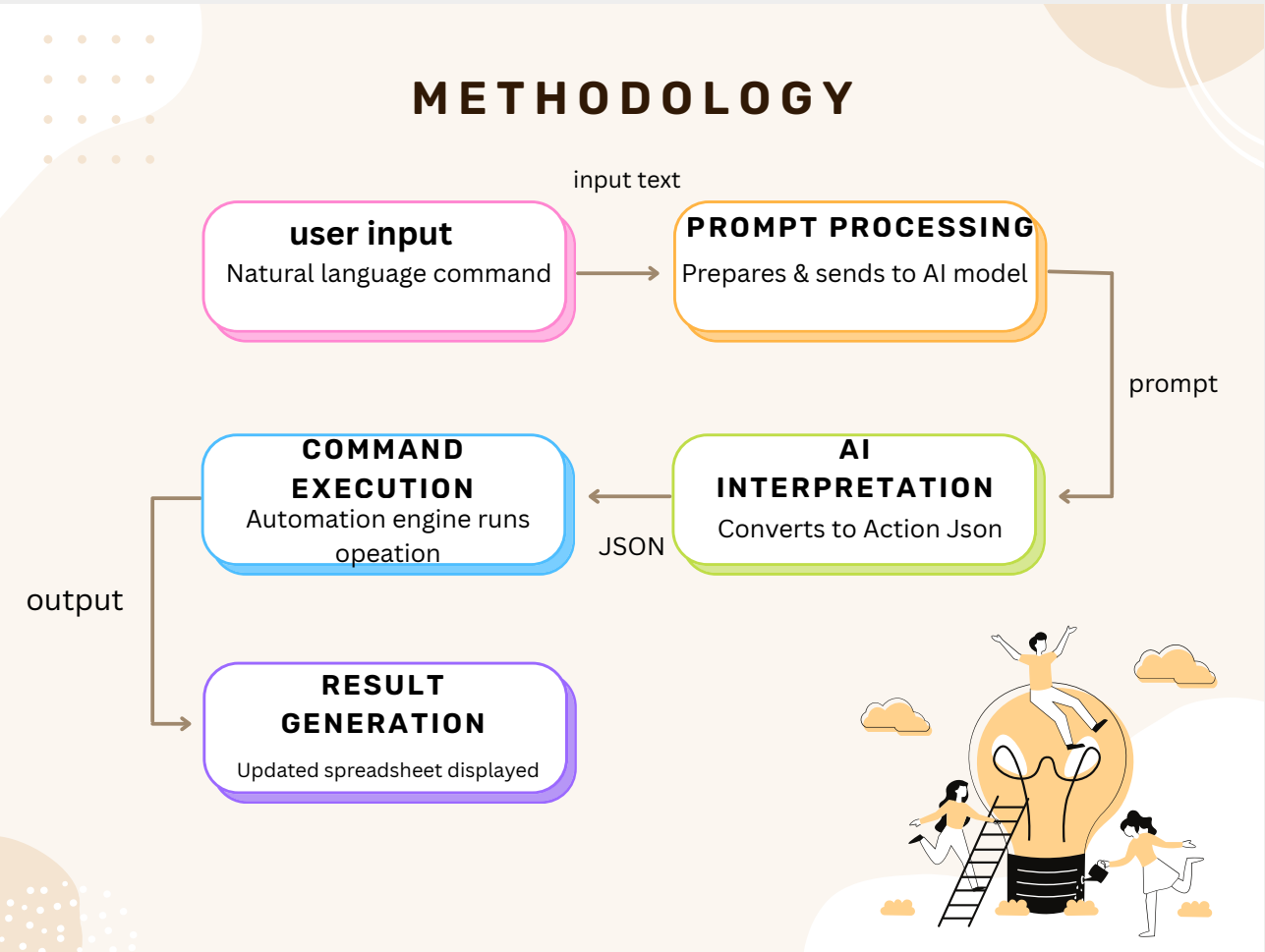
5.1 Agentic AI Loop Design

The PyXcel system follows an **agentic AI design pattern**, where the AI behaves more like an intelligent assistant rather than a simple chatbot. Instead of only answering questions, the system first examines the spreadsheet environment, understands the current data, decides what

action should be taken, performs the action, and then reports the result to the user. This process follows a loop consisting of four stages: **Observe, Reason, Act, and Report**. During the observe stage, the system gathers information from the workbook. In the reasoning stage, the AI model analyzes the instruction and decides the appropriate operation. In the action stage, the generated code or formula is executed, and finally, the report stage provides a clear and human-readable output to the user.

5.2 Prompt Engineering Strategy

To ensure reliable results, PyXcel uses carefully designed prompts when communicating with the AI model. Each feature in the system has a specific prompt format that controls the type of response generated. For example, when generating Python code for tasks such as macro replacement or data cleaning, the prompt instructs the model to output only plain Python code without explanations. For KPI generation, the model is required to produce results in a structured JSON format so that the interface can display them properly. Similarly, for formula



generation, the output is restricted to a valid Excel formula beginning with the "=" symbol.

5.3 Context Injection

An important part of the PyXcel methodology is the **context injection mechanism**. Before sending any instruction to the AI model, the system first analyzes the Excel workbook. A module called the Workbook Inspector reads the file, identifies sheet names, column headers, row counts, data types, and a few sample rows. This information is converted into a structured context description and attached to the prompt. By doing this, the AI model has a clear understanding of the actual spreadsheet data it is working with.

5.4 Safe Code Execution

The Python code generated by the AI is executed in a controlled environment using Python's `exec()` function. Only necessary objects such as the workbook and file path are made available within the execution scope. Access to other system resources such as external files, networks, or system commands is restricted. This ensures that the generated code runs safely without affecting the rest of the system.

5.5 Threaded Background Processing

To keep the user interface responsive, PyXcel performs all AI processing tasks in background threads. The GUI developed with PySide6 uses worker threads so that when a request is sent to the AI model through Ollama, the interface does not freeze. While the model processes the request, users can continue interacting with the application.

5.6 Conversational Memory

The chat feature in PyXcel supports conversational interaction with spreadsheet data. The system maintains a short history of recent messages within the session. This conversation history is included in new requests so that the AI model can understand the context of previous questions. As a result, users can ask follow-up questions and receive more meaningful responses based on earlier interactions.

6. Features of the System

6.1 Natural Language Macro Replacement

This feature allows users to perform spreadsheet operations using simple natural language instructions instead of writing macros or code. For example, a user can type a command such as highlighting rows where profit is negative. The system interprets the instruction and generates Python code using the **openpyxl** library to perform the task directly on the workbook. The generated code is also shown to the user for transparency.

6.2 AI Formula Generator

The system can automatically generate Excel formulas based on user instructions written in plain English. Instead of manually creating complex formulas, users can describe the required calculation, and the AI model generates the correct formula using modern Excel functions such as XLOOKUP, FILTER, and SUMPRODUCT. This makes formula creation easier for beginners.

6.3 Intelligent Data Cleaning

Data cleaning is another important feature of the system. Users can describe cleaning tasks in natural language, such as removing duplicate rows, filling missing values, or formatting data. The system generates Python code using

the pandas library to perform these operations. After processing, the cleaned data is written back to the Excel file, and the system reports the changes made.

6.4 Conversational Data Analysis

The application includes a chat-based interface that allows users to ask questions about their spreadsheet data. The AI model analyzes the workbook and provides answers related to totals, averages, comparisons, trends, or data issues. The conversation history is maintained so that users can ask follow-up questions and receive context-aware responses.

6.5 Automatic KPI Card Generation

The system can automatically identify important Key Performance Indicators (KPIs) from the uploaded spreadsheet. It analyzes the data columns, calculates relevant metrics, and displays them as KPI cards in the interface. Each card shows the metric name, calculated value, and a brief explanation.

6.6 Workbook Structure Inspector

A background module continuously examines the structure of the Excel workbook. It collects information such as sheet names, column headers, row counts, and sample data. This information is used as context for the AI model, ensuring that all responses are based on the actual spreadsheet data.

6.7 One-Click Launcher

To make the system easy to use, a one-click launcher is provided through a batch script. This script automatically checks the status of the AI environment, starts the backend server built with Flask, and launches the graphical interface developed using PySide6. As a result, users can start the entire system with a simple double-click.

7. Technologies Used

Technology	Version	Purpose
Python	3.11+	Core programming language used for backend development and AI integration.
PySide6	6.x	Qt-based graphical user interface framework used to build the desktop application.
Ollama	Latest	Local LLM runtime that serves the AI model on localhost:11434.
LLaMA 3.1	8B / 70B	Open-source large language model used for natural language understanding and code generation.
Flask	3.x	Lightweight REST API server that connects the GUI with the AI backend.
openpyxl	3.x	Library used to read and write Excel .xlsx files and apply styles, formulas, and formatting.
pandas	2.x	Library used for data processing, cleaning, and transformation using DataFrame operations.
requests	2.x	HTTP client used for communication between the GUI, backend, and AI services.
flask-cors	4.x	Enables Cross-Origin Resource Sharing for accessing the local API.
QThread	PySide6	Used for background processing to keep the GUI responsive during AI model inference.

Category	Approximate Percentage
Companies using Excel / spreadsheets	50–70%
Companies that actually use macros	10–20%
Companies heavily depending on macros	5–10%

so the websites that provide the reports are:

1. IndustryResearch.biz
2. ProcureInsights.com
3. CorporateComplianceInsights.com

- Report page: spreadsheet software market report
- States that about 72% of enterprises use spreadsheets for planning and business intelligence.

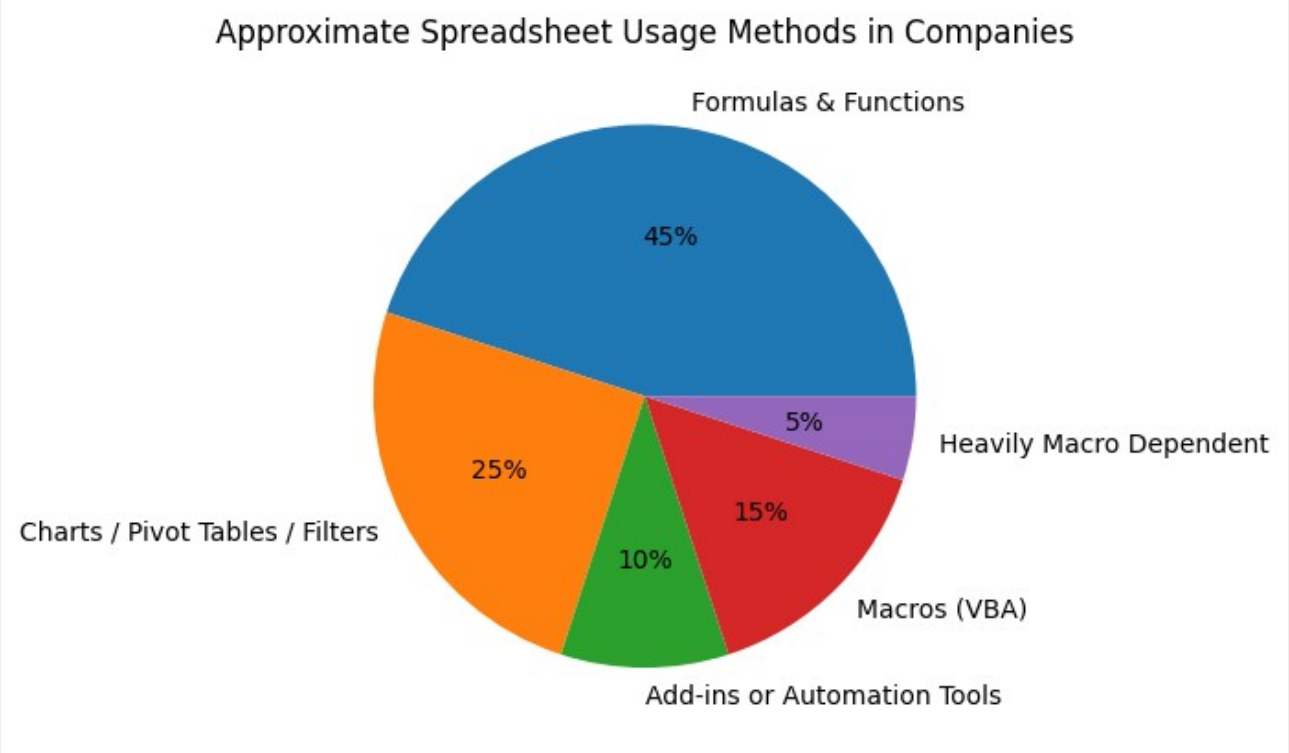
Website name you can cite:
IndustryResearch.biz

1 Spreadsheet Software Market Report

Website: Industry Research

- Report page: spreadsheet software market report
- States that about 72% of enterprises use spreadsheets for planning and business intelligence.

Website name you can cite:



IndustryResearch.biz

2 Procurement Insights Report

Website: Procure Insights

-
- Report says about 85–90% of professionals still use spreadsheets in organizations.

Website name:
ProcureInsights.com

3 Corporate Compliance Insights Study

Website: Corporate Compliance Insights

- Study found about 50% of companies rely on spreadsheets for auditing and controls.

Website name:
CorporateComplianceInsights.com

1 "Spreadsheets in Financial Departments: An Automated Analysis of 65,000 Spreadsheets using the Luminous Technology"

- Authors: Kevin McDaid et al.
- Published on arXiv research repository.
- The study analyzed 65,000 spreadsheets from organizations and examined the presence of macros and automation in real corporate spreadsheets.

This is one of the most cited academic studies about spreadsheet and macro usage in companies.

2 VBA Security Tools Market Research Report 2033

- Website: Growth Market Reports
- Focuses on enterprise environments that use VBA macros and security tools in organizations.

3 Studies referenced in Excel automation articles

Some technical articles and surveys report that around 68% of professionals use macros for spreadsheet automation tasks.

According to reports and research papers, there are alternatives to macros that allow similar "one-click automation" in spreadsheets.

- Microsoft Power Automate
- No-code automation tool used with Microsoft Excel.

- Users can trigger workflows with **one click or automatically** (e.g., update data, send reports).

  Office Scripts

- Modern scripting feature in Excel for the web.
- Allows **button-based automation similar to VBA macros**.

  Google Apps Script

- Used in **Google Sheets**.
- JavaScript-based scripts can run **with a custom button click**.

  Power Query

- Allows **one-click data import, cleaning, and refresh** without macros.**Microsoft Power Automate**
- No-code automation tool used with **Microsoft Excel**.
- Users can trigger workflows with **one click or automatically** (e.g., update data, send reports).

  Office Scripts

- Modern scripting feature in Excel for the web.
- Allows **button-based automation similar to VBA macros**.

  Google Apps Script

- Used in **Google Sheets**.
- JavaScript-based scripts can run **with a custom button click**.

  Power Query

- Allows **one-click data import, cleaning, and refresh** without macros.

  Python integration

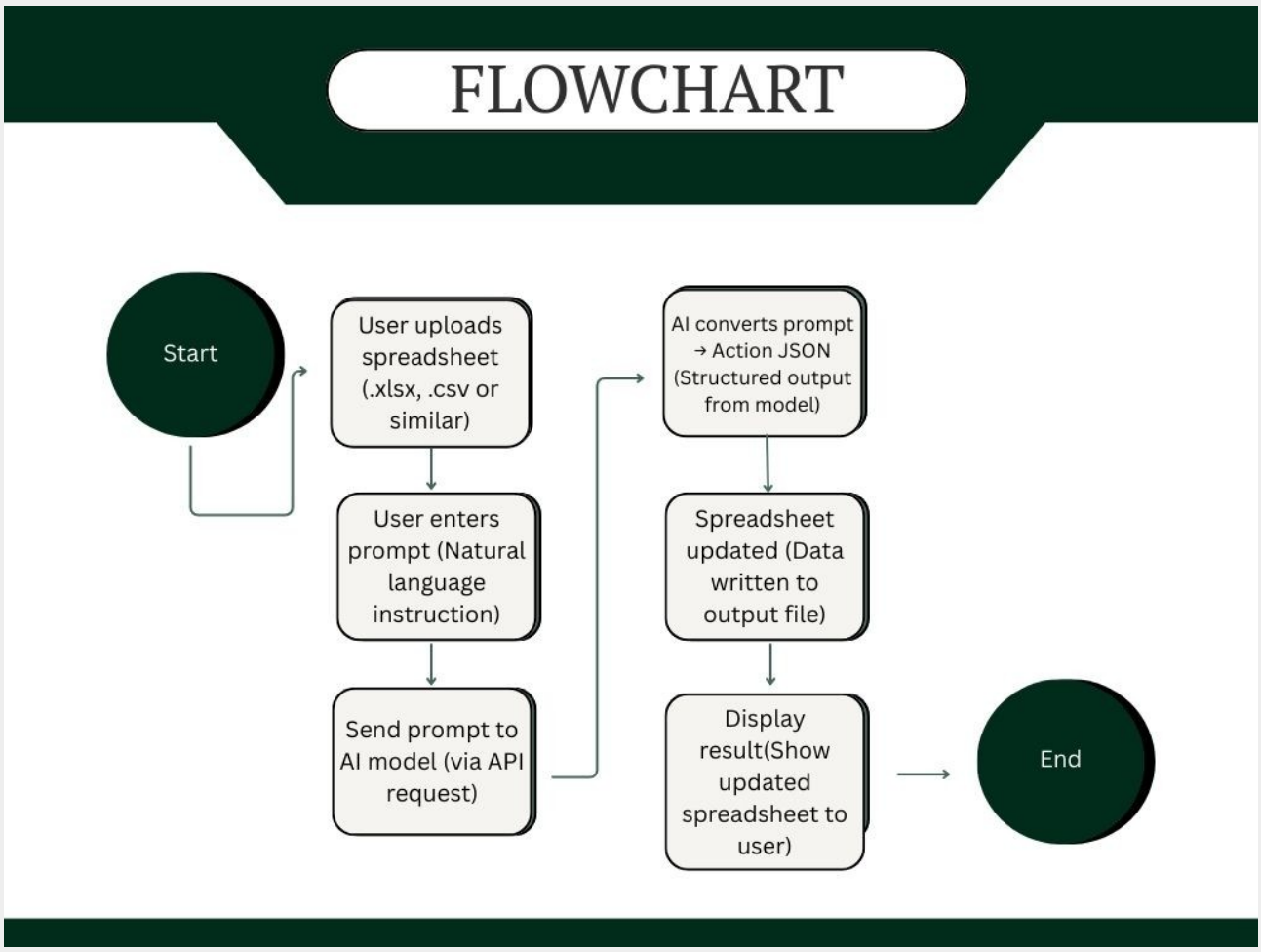
- Used in modern Excel versions for **automated data analysis and reporting**.

  Python integration

- Used in modern Excel versions for **automated data analysis and reporting**.

These alternatives to **Visual Basic for Applications** macros also have some **disadvantages**. Briefly:

Alternative Tool	Disadvantages
Microsoft Power Automate	Requires internet connection, some features need paid subscription, slower for heavy spreadsheet calculations
Office Scripts	Works mainly in Excel web version, limited features compared to VBA, not supported in older Excel versions
Google Apps Script	Works only in **Google Sheets**, requires JavaScript knowledge, limited integration with desktop Excel
Power Query	Mainly for data cleaning/import, cannot automate every UI task like macros
Python integration	Requires programming knowledge, setup and libraries needed, not as simple as one-click macro for basic users



8. Advantages

8.1 Complete Offline Operation

One of the major advantages of PyXcel is that it can operate completely offline after the initial setup. Once the required AI model is downloaded, the system performs all tasks such as data analysis, AI inference, and file processing directly on the user's local computer. This makes the system suitable for environments where internet access is limited or restricted.

8.2 Zero Data Privacy Risk

PyXcel ensures strong data privacy because all spreadsheet processing takes place locally. The user's files are never uploaded to external servers. This is especially useful for organizations that handle sensitive information such as

financial records, personal data, or confidential business documents.

8.3 No Subscription Cost

Another advantage of the system is that it does not require any subscription fees or API payments. The AI model used in the system is open-source, and the system runs locally using tools such as Ollama. Since the project is built using the open-source Python ecosystem, the overall operating cost of the system remains very low.

8.4 Adaptability to Spreadsheet Changes

Unlike traditional macros that may stop working when the structure of a spreadsheet changes, PyXcel automatically examines the workbook before performing any operation. If column positions, sheet names, or row counts change, the system adjusts its generated code accordingly.

8.5 Transparency and Easy Verification

The system displays all generated code in the results panel. This allows users to see exactly what operations are being performed on their data. Such transparency helps users understand the process and builds trust in the system's operations.

8.6 Smooth Migration to the New Interface

PyXcel is designed in a way that allows a gradual transition from the older Tkinter-based interface to the new interface developed with PySide6. Existing backend modules remain unchanged, allowing users to adopt the new system without affecting their current workflows.

9. Limitations

9.1 Hardware Requirements

One of the main limitations of the system is the hardware requirement for running the AI model locally. The LLaMA 3.1 model needs a computer with sufficient memory and processing power. At least 8 GB of RAM is required, while 16 GB is recommended for smoother performance. On systems with lower specifications, the response time for AI queries may become slow.

9.2 Code Execution Risk

The system executes Python code generated by the AI using the `exec()` function. Although the execution environment is restricted to the workbook and required variables, there is still a possibility that the AI may generate incorrect or unintended code, especially for complex instructions. Future versions plan to include additional safety features such as code review and undo options.

9.3 Model Context Limitations

Another limitation arises when working with very large spreadsheets containing many columns or sheets. The AI model can only process a limited amount of information at a time. To manage this, the system currently includes only a few sample rows from each sheet as context, which may sometimes limit the accuracy of complex data analysis.

9.4 Formula Generation Accuracy

Although the AI model has good knowledge of Excel formulas, it may occasionally produce incorrect formulas, especially when dealing with advanced or version-specific functions. This issue may occur in older versions of Excel that do not support modern functions.

9.5 Single File Processing

At present, the system processes only one Excel file at a time during a session. Operations that require combining

or comparing multiple spreadsheets must be performed manually using the existing backend tools. Automated multi-file processing through AI is planned for future versions.

9.6 Platform Dependency

The current launcher script used to start the system is designed mainly for Windows environments. Users working on macOS or Linux systems may need to start the required services manually through the command line. Future updates aim to provide a launcher that supports multiple operating systems.

10. Future Enhancements

The PyXcel system can be further improved by adding new features and expanding its capabilities in future versions. These improvements are planned in three stages: short-term, medium-term, and long-term developments.

10.1 Short-Term Enhancements (Version 2.1)

In the next release, the system will include an **undo and redo feature** that allows users to quickly reverse or restore AI-generated changes in the spreadsheet. A **pre-execution review panel** will also be added so that users can check and edit the generated code before it runs. To improve usability across different operating systems, new launcher scripts will be created for macOS and Linux systems. In addition, the system will support exporting files in formats such as CSV and Google Sheets along with Excel files. Another planned feature is **inline formula insertion**, where users can generate formulas and insert them directly into a selected cell.

10.2 Medium-Term Enhancements (Version 3.0)

In the medium term, PyXcel will support **AI-driven operations across multiple Excel files**, allowing users to

merge, compare, or analyze data from several workbooks at once. The system will also be able to create **charts and visualizations** automatically from natural language instructions. Another planned feature is **scheduled automation**, where users can set tasks to run automatically when a file is opened or at a specific time. The interface will also allow users to choose between different AI models supported by Ollama, such as LLaMA or other models. Additionally, a **plugin system** will be introduced so that developers can add custom tools and functions that the AI agent can use.

10.3 Long-Term Enhancements (Version 4.0)

In the long term, PyXcel aims to provide a **complete spreadsheet co-pilot experience**. The AI system will monitor user actions and automatically suggest improvements, detect errors, and highlight unusual patterns in real time. Future versions may also include **voice command support** using speech-to-text technology, enabling hands-free spreadsheet interaction. Specialized AI models could also be developed for specific domains such as finance, healthcare, or supply chain management. Furthermore, collaborative features may be added to allow multiple users to work on the same workbook, with AI helping to manage conflicts and maintain data consistency. Integration with local databases will also enable the system to generate SQL queries and import data directly into spreadsheets.

11. Conclusion

PyXcel represents a meaningful advancement in how non-technical users interact with spreadsheet data. By replacing brittle, opaque VBA macros with natural language instructions processed by a locally hosted LLM, the system democratizes spreadsheet automation without sacrificing privacy, cost efficiency, or data control. The integration of Ollama and LLaMA 3.1 as the AI backbone delivers a genuinely capable natural language interface that can

generate executable code, produce Excel formulas, clean data, answer analytical questions, and surface business KPIs — all from plain-English commands, all offline, and all transparently. The PySide6 architectural foundation provides a modern, maintainable GUI platform that cleanly separates presentation from business logic, enabling PyXcel's existing Python backend modules to be progressively migrated without disruption. The three-tier architecture (GUI → Flask API → Ollama) is modular, testable, and extensible. While current limitations around hardware requirements, code execution safety, and model context window size represent real engineering challenges, each has a clear mitigation pathway in the planned roadmap. The system is production-ready for its defined use case and delivers immediate value to users seeking to escape the macro maintenance cycle. The future of spreadsheet interaction is conversational, contextual, and locally intelligent. PyXcel is a concrete realization of that future, built today, running offline, and costing nothing to operate.